



中国科学技术大学

写好自己的头文件 ——从使用者到设计者

徐 伟

E-mail: xuweihf@ustc.edu.cn



课程概述

- ❖ 头文件的本质与作用
- ❖ 头文件里面有哪些内容
- ❖ 为什么要自己编写头文件
- ❖ 动手写自己的头文件
- ❖ 多文件工程中头文件的作用



头文件的本质与作用

- 学习过C语言的都熟悉这样一段代码：

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    return 0;
```

```
}
```

<stdio.h> 到底是什么呢？

<stdio.h> 和 "stdio.h" 这两种写法皆可行吗？为什么？这二者有何区别呢？



头文件的本质与作用

- 回顾过去：一直在 `#include <stdio.h>`。我们像“使用者”，调用别人写好的函数（如 `printf`, `scanf`）。
- 问题：如果我想写一个“计算平均分”的函数，给别的程序用，或者给团队协作用，该怎么办？

我们要从“调用者”变成“提供者”



头文件的本质与作用

头文件的本质

- ❖ 头文件（.h）是接口，源文件（.c）是实现。
- ❖ 头文件 = 对外承诺（告诉可以提供什么功能，叫什么名字，需要什么参数，但没告诉具体实现）
- ❖ 源文件 = 内部处理（具体的实现步骤）。

编译视角

- ❖ 头文件是声明，源文件是定义
- ❖ 编译器在编译时只需要知道**头文件**就能通过检查，链接器再把**源文件**和**调用者**连起来



头文件的本质与作用

1、include的作用

- 预处理过程：在include的地方，把头文件里的内容原封不动的复制到引用该头文件的地方

2、头文件的引用

- 头文件引用有两种形式：`#include <stdio.h>` 和 `include "main.h"`。
- 用`< >`引用的一般是编译器提供的头文件，编译时会在指定的目录中去查找头文件（具体是哪个目录，在编译器路径中设置）。用`" "`引用的一般是个人写的头文件

总结：系统提供的头文件用`< >`引用，自己写的用`" "`引用



头文件里面有哪些内容

头文件里一般包括宏定义， 全局变量， 函数原型声明

❖ **函数声明：** 告诉编译器有这个函数存在， 如： `int add(int a, int b);`

```
// math_utils.h
```

```
#ifndef __MATH_UTILS_H
```

```
#define __MATH_UTILS_H
```

```
// 函数声明
```

```
int add(int a, int b);
```

```
int multiply(int a, int b);
```

```
#endif
```



头文件里面有哪些内容

- ❖ **宏定义：** 常量或通用代码片段。可以在头文件中定义一些宏，用于常量表达式、条件编译或代码优化。

```
#define PI 3.14159
```

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```




头文件里面有哪些内容

- ❖ **数据类型定义**：头文件中可以使用 typedef 定义新的数据类型，也可以定义结构体或枚举。

// 定义新类型

```
typedef unsigned int uint;
```

// 定义结构体

```
typedef struct {
```

```
    int x;
```

```
    int y;
```

```
} Point;
```

// 定义枚举

```
typedef enum {
```

```
    RED,
```

```
    GREEN,
```

```
    BLUE
```

```
} Color;
```



头文件里面有哪些内容

- ❖ **全局变量声明**：在头文件中声明全局变量，但其定义应放在对应的 .c 文件中。

//使用 extern 关键字，完成跨文件共享全局变量
假设有两个文件

//a.c

int global_count = 100; // 定义全局变量

//b.c

extern int global_count; // 声明：这个变量在别处

void func() {

global_count++; // 可以用

}



头文件里面有哪些内容

- ❖ **内联函数（C99 及以上版本）**：头文件可以包含内联函数的定义，这种函数通常体积小、性能高，直接在调用处展开。

```
static inline int square(int x)
{
    return x * x;
}
```



头文件里面有哪些内容

- ❖ 头文件可以包含内联函数，C 标准库提供了一系列常用的头文件，包含许多函数和宏，方便程序开发。以下是一些常见的标准头文件：

头文件

描述

<stdio.h>

标准输入输出（如 printf、scanf）

<stdlib.h>

通用工具（如内存分配、随机数生成）

<string.h>

字符串操作（如 strcpy、strlen）

<math.h>

数学函数（如 sin、sqrt）

<time.h>

时间和日期操作

<ctype.h>

字符处理（如 isalpha、isdigit）



为什么要自己写头文件

- **代码复用**

- 没有头文件时需要把一堆函数复制粘贴到每个新文件里
- 有头文件后，只需要 `#include "my_math.h"`。

- **模块化与协作**

- A同学写 `math_tools.c`，B同学写 `string_tools.c`。
- 只需要提供 `.h` 文件给队友，队友就能调用你的函数，不需要看你复杂的实现代码。

- **保护源码**

团队可以给你提供 `.h` 和编译好的 `.lib/.o` 文件，你不需要知道源代码也能开发。



动手写第一个头文件

❖ 头文件格式说明

#ifndef 头文件名 //头文件名的格式为"_头文件名_",
注意要大写

#define 头文件名

头文件内容

#endif



动手写第一个头文件

❖ 目标： 创建一个简单的数学工具库

1. 创建 my_math.h

```
#ifndef MY_MATH_H
#define MY_MATH_H

// 函数声明
int max_of_two(int a, int b);
int min_of_two(int a, int b);

#endif // MY_MATH_H
```

2. 创建 my_math.c:

```
#include "my_math.h"

int max_of_two(int a, int b) {
    return (a > b) ? a : b;
}

int min_of_two(int a, int b) {
    return (a < b) ? a : b;
}
```



动手写第一个头文件

3. 在 main.c 中调用自定义函数

```
#include <stdio.h>
```

```
#include "my_math.h" // 引用自己写的头文件（用双引号）
```

```
int main() {  
    int a = 10, b = 20;  
    printf("最大值: %d\n", max_of_two(a, b));  
    printf("最小值: %d\n", min_of_two(a, b));  
    return 0;  
}
```




动手写第一个头文件

编译时，需要将头文件的 `.c` 文件与主程序一起编译并链接：

- 错误示范： `gcc main.c` （会报错，找不到函数定义）
- 正确示范： `gcc main.c my_math.c -o output`



.h与.c职责边界

.h文件：对外接口

- ▶ 类型定义 (typedef, struct)
- ▶ 宏常量 (#define)
- ▶ 函数声明
- ▶ 必要的注释说明

.c文件：内部实现细节

- ▶ 内部函数
- ▶ 内部数据
- ▶ 算法与优化
- ▶ 内部实现



多文件工程中的头文件

❖ 头文件在大型工程中的作用

1. 模块解耦

- 目标源文件不需要知道 其他源文件 里排序的具体算法，只需要 `#include "score.h"` 就能调用 排序函数

2. 依赖管理

- 头文件通过 `#include` 明确表达了模块间的依赖关系
- 例如 `score.h` 包含 `student.h`，表示成绩模块依赖于学生结构体

3. 分工协作

- 团队中不同成员可以分别开发 `student.c`、`score.c` 等模块，只需事先约定好头文件中的接口，就可以并行开发，最后整合编译。



多文件工程中的头文件

- ❖ 问题： 在大型项目中，头文件可能会被多次包含。为避免这种问题，头文件通常使用**头文件保护机制**
- ❖ 解决方案1： 预编译宏（头文件守卫）

错误写法（可能报错）

```
// my_math.h  
int add(int a, int b);  
int sub(int a, int b);
```

正确写法（使用守卫）

```
#ifndef MY_MATH_H  
#define MY_MATH_H  
int add(int a, int b);  
int sub(int a, int b);  
#endif
```

原理： 第一次包含时定义宏，第二次包含时检查到宏已定义，直接跳过内容。



多文件工程中的头文件

❖ 解决方案2: `#pragma once` (非标准但常用)

使用 `#pragma once` 指令也是一种防止重复包含的方式, 且更简洁

```
#pragma once
```

```
// 头文件内容
```



多文件工程中的头文件

项目结构设计

❖ 1. 项目目录结构

- 在大型项目中，合理组织头文件是实现模块化设计、团队协作和代码复用的关键。
- 头文件的组织需要遵循一定的规则，以确保项目的结构清晰、依赖关系明确，并避免重复包含和命名冲突的问题。



多文件工程中的头文件

在大型项目中，通常将头文件和源文件按照模块或功能分类，并将头文件放在一个专门的目录下。例如：

```
1  MyProject/
2  |— include/           // 头文件目录
3  |   |— module1/       // 模块1相关头文件
4  |   |   |— module1.h
5  |   |   |— utils1.h
6  |   |— module2/       // 模块2相关头文件
7  |   |   |— module2.h
8  |   |   |— utils2.h
9  |   |— common/        // 公共头文件
10 |   |   |— config.h
11 |   |   |— macros.h
12 |— src/               // 源文件目录
13 |   |— module1/
14 |   |   |— module1.c
15 |   |   |— utils1.c
16 |   |— module2/
17 |   |   |— module2.c
18 |   |   |— utils2.c
19 |   |— main.c
20 |— build/             // 编译输出目录
21 |— Makefile           // 构建脚本
22 |— README.md          // 项目说明文件
```



多文件工程中的头文件

❖ 2. 头文件命名规则

模块化命名：头文件应以模块命名，避免与其他模块或标准库头文件冲突。例如：

- module1.h 表示模块1的主头文件。
- utils1.h 表示模块1的工具函数头文件。
- 公共头文件：将项目中的全局配置、宏、数据类型定义等公共内容放在 common/ 目录下。



多文件工程中的头文件

❖ 头文件的内容组织

❖ 1. 主模块头文件

- ☞ 主要提供模块的外部接口声明，供其他模块使用。（系统提供的头文件通常以 `_` 开头，自己写的头文件通常以 `__` 开头，防止重复定义）只包含必要的内容，隐藏模块内部实现细节。



多文件工程中的头文件

❖ 头文件的内容组织

❖ 1. 主模块头文件

```
// module1.h
#ifndef __MODULE1_H
#define __MODULE1_H
#include <stdio.h> // 标准库头文件
#include "common/config.h" // 项目公共头文件

// 模块1对外的函数声明
void module1_init();
void module1_process();

#endif // __MODULE1_H
```



多文件工程中的头文件

❖ 头文件的内容组织

❖ 2. 工具函数头文件

定义模块内部使用的工具函数或辅助功能，通常不直接对外暴露。

```
// utils1.h
#ifndef __UTILS1_H
#define __UTILS1_H

// 模块1内部工具函数
int helper_function(int a, int b);

#endif // __UTILS1_H
```



多文件工程中的头文件

❖ 头文件的内容组织

❖ 3. 公共头文件

定义整个项目的全局配置、数据类型、宏和其他公共内容。
这些头文件通常被多个模块共享。

```
// config.h
#ifndef __CONFIG_H
#define __CONFIG_H
// 项目全局配置
#define MAX_BUFFER_SIZE 1024
#define PROJECT_NAME
"MyProject"
#endif // __CONFIG_H
```



多文件工程中的头文件

❖ 头文件的相互引用

❖ 避免循环依赖

在大型项目中，头文件可能会相互包含，导致循环依赖问题（A.h 引用 B.h，而 B.h 又引用 A.h）。为避免此问题：

- 使用前向声明（Forward Declarations）代替直接包含头文件
- 仅在需要完整类型定义时包含相关头文件。



多文件工程中的头文件

❖ 头文件的相互引用

❖ 避免循环依赖

```
// module1.h  
struct Module1{ ... }
```

```
// module2.h  
// 使用前向声明避免包含 module1.h  
struct Module1;  
// 模块2对外接口  
void module2_process(struct Module1* module1_instance);
```



多文件工程中的头文件

❖ 前向声明

☞ 函数、结构体

`struct Node; // 前向声明，告诉编译器"Node 这个结构体存在"`

```
struct List {  
    struct Node *head; // 可以用指针，因为指针大小已知  
                        // 若不是指针会报错吗?  
};
```

```
struct Node { // 后面再完整定义  
    int data;  
    struct Node *next;  
};
```



多文件工程中的头文件

❖ 前向声明

❧ 不用前向申明

```
#include <stdio.h>
```

```
int main() {
```

```
    print_hello("World"); // 可以先调用
```

```
    return 0;
```

```
}
```

```
void print_hello(char *name) { // 实际定义在后面
```

```
    printf("Hello %s\n", name);
```

```
}
```

直接用**gcc**编译的结果如何？



多文件工程中的头文件

❖ 为什么可以编译通过，编译器处理流程

- ☞ 看到 `print_hello("World")`，没见过声明
 - ☞ 自动隐式声明：`int print_hello(char *)`; (参数从 "World" 推断为 `char *`)
 - ☞ 编译通过，链接时找到真正的 `print_hello` 函数
 - ☞ 实际定义是 `void` 返回，但编译器已经按 `int` 处理了
- ❖ ——这就埋下了隐患

在**C89/C90**标准及更早的版本中，如果编译器遇到一个没见过声明的函数调用，它会自动推断这个函数：

返回值：默认 `int`

参数：从你传入的参数推断类型和数量



多文件工程中的头文件

❖ 前向声明例子2

```
int main() {  
    float result = square(3.14);  
    return 0;  
}
```

```
double square(double x) {  
    return x * x;  
}
```

结果如何?



多文件工程中的头文件

- ❖ 代码能编译通过，是因为编译器默认补了个声明。这不是好习惯，风险有：
 - ❧ 返回值类型错误导致程序崩溃
 - ❧ 参数类型不匹配导致数据错乱
 - ❧ 编译时只能当警告看，运行时可能出诡异 bug



总结

在项目中，头文件的组织是非常重要的环节之一：

- 模块化设计：每个模块有自己的头文件，头文件只暴露必要的接口
- 避免重复包含：使用头文件保护（`#ifndef...#define` 或 `#pragma once`）
- 减少头文件依赖：使用前向声明避免不必要的头文件包含
- 按需包含：仅在需要的源文件中包含头文件，避免头文件之间的过度耦合



谢谢!



中国科学技术大学

University of Science and Technology of China